

Vue.js Masterclass Notes

Comprehensive Study Guide & Architecture Analysis based on Elzero Web School

1. Introduction & Core Features

Vue.js was created by **Evan You** after working at Google on Angular projects. His goal was to extract the lightweight, reactive, and elegant parts of Angular to build a flexible frontend framework.

Key Framework Features:

- **Lightweight:** Extremely small bundle size (~20-30KB gzipped), ensuring fast initial load times.
- **Virtual DOM:** A lightweight in-memory representation of the real DOM. Vue calculates minimal DOM updates using an optimized diffing algorithm.
- **Reusable Components:** Encourages modular development by isolating UI into self-contained units (Templates, Scripts, Styles).
- **Directives:** Special HTML attributes prefixed with `v-` that apply reactive behavior to elements.
- **Event Handling:** Streamlined listener attachment using `v-on` syntax with event modifiers.
- **Animations:** Built-in `<Transition>` wrapper component for smooth entry/exit transitions.
- **Watchers:** Reactive observers that run custom side-effect logic whenever target data changes.
- **Vue CLI & Router:** Complete ecosystem tooling for scaffolded SPA projects and client-side routing.
- **Unit Testing:** Native ecosystem integration with modern test runners (Vitest, Jest) for component testing.
- **Mixins:** Legacy code distribution strategy (superseded by Composition API in Vue 3).



FIGURE 1: VUE VIRTUAL DOM UPDATE WORKFLOW

2. Architecture: Single Page Applications (SPA), MVC & MVVM

Single Page Application (SPA)

In an SPA, the browser loads a single `index.html` page. When a user navigates or interacts with the app, JavaScript fetches data dynamically via APIs and updates the DOM in place without triggering full page reloads.

MVC Architecture

Model-View-Controller: Classic backend pattern where Controller mediates input between View and Model.

- **Model:** Data structure & database logic.
- **View:** User UI interface (HTML/Templates).
- **Controller:** Processes requests & updates Model/View.

MVVM Architecture (Vue's Pattern)

Model-View-ViewModel: Modern frontend reactive architecture.

- **Model:** JS Objects / Reactive State.
- **View:** The DOM / Rendered HTML template.
- **ViewModel:** Vue's core instance binding Model and View bi-directionally.



FIGURE 2: VUE.JS MVVM ARCHITECTURE DATA FLOW

3. Detailed Analysis of Vue Directives

1. v-text

Updates the element's text content. It overwrites any existing child nodes inside the element. Equivalent to mustache interpolation `{{ }}`.

```
<!-- Using Directive -->
<span v-text="username"></span>

<!-- Equivalent Mustache Syntax -->
<span>{{ username }}</span>
```

2. v-html

Renders raw HTML string inside an element. It replaces the inner HTML of the target node.

```
<template>
  <div v-html="formattedBio"></div>
</template>

<script>
export default {
  data() {
    return {
      formattedBio: '<strong>Software Engineer</strong> based in Egypt.'
    }
  }
}
</script>
```

Security Warning: Dynamically rendering arbitrary HTML can lead to Cross-Site Scripting (XSS) vulnerabilities. Only use `v-html` on trusted content, never on user-provided inputs.

3. v-for

Renders a list of items based on an Array or Object. Always couple it with a unique `:key` attribute to maintain component state and enable DOM optimizations.

```

<template>
  <ul>
    <li v-for="(skill, index) in skills" :key="index">
      {{ index + 1 }}. {{ skill }}
    </li>
  </ul>
</template>

<script>
export default {
  data() {
    return {
      skills: ['Vue.js', 'JavaScript', 'HTML5/CSS3', 'Tailwind CSS']
    }
  }
}
</script>

```

4. Conditional Rendering (v-if, v-else-if, v-else)

Conditionally renders elements. If the condition is false, elements are removed entirely from the DOM tree.

```

<template>
  <div class="status-container">
    <p v-if="score >= 90">Grade: Excellent</p>
    <p v-else-if="score >= 50">Grade: Passed</p>
    <p v-else>Grade: Failed</p>
  </div>
</template>

```

Rule: `v-else` and `v-else-if` elements must immediately follow a `v-if` or `v-else-if` element without any intervening HTML elements or comments.

5. v-once

Renders the element and component once only. On subsequent re-renders, the element/component and all its children will be treated as static content and skipped. This optimizes update performance.

```

<template>
  <!-- This title will not change even if 'title' data changes -->
  <h1 v-once>{{ title }}</h1>
  <button @click="title = 'Updated Title'">Change Title</button>
</template>

```

6. v-show

Toggles the element's visibility by modifying its inline CSS `display` property (e.g., `display: none;`). The element remains in the DOM tree at all times.

```
<template>
  <div v-show="isVisible" class="modal">
    <p>This modal is toggled via CSS display property.</p>
  </div>
</template>
```

4. Comparison: v-if vs. v-show

Feature / Aspect	v-if Directive	v-show Directive
DOM Behavior	Physically adds/removes elements from DOM	Keeps element in DOM, toggles <code>display: none</code>
Initial Render Cost	Low (If false, does not render element)	Higher (Always renders regardless of condition)
Toggle Cost	High (Destroys and recreates DOM nodes)	Very Low (Only updates CSS styling property)
Template Support	Supports <code><template></code> tag wrapper	Does NOT support <code><template></code> tag
Best Use Case	Conditions that rarely change at runtime	Elements requiring frequent visibility toggling